

Logical and Physical Database Design

Claire Trebing, Jason Young, Illya Starikov

Due Date: March 18, 2016

1 Revised Problem Statement

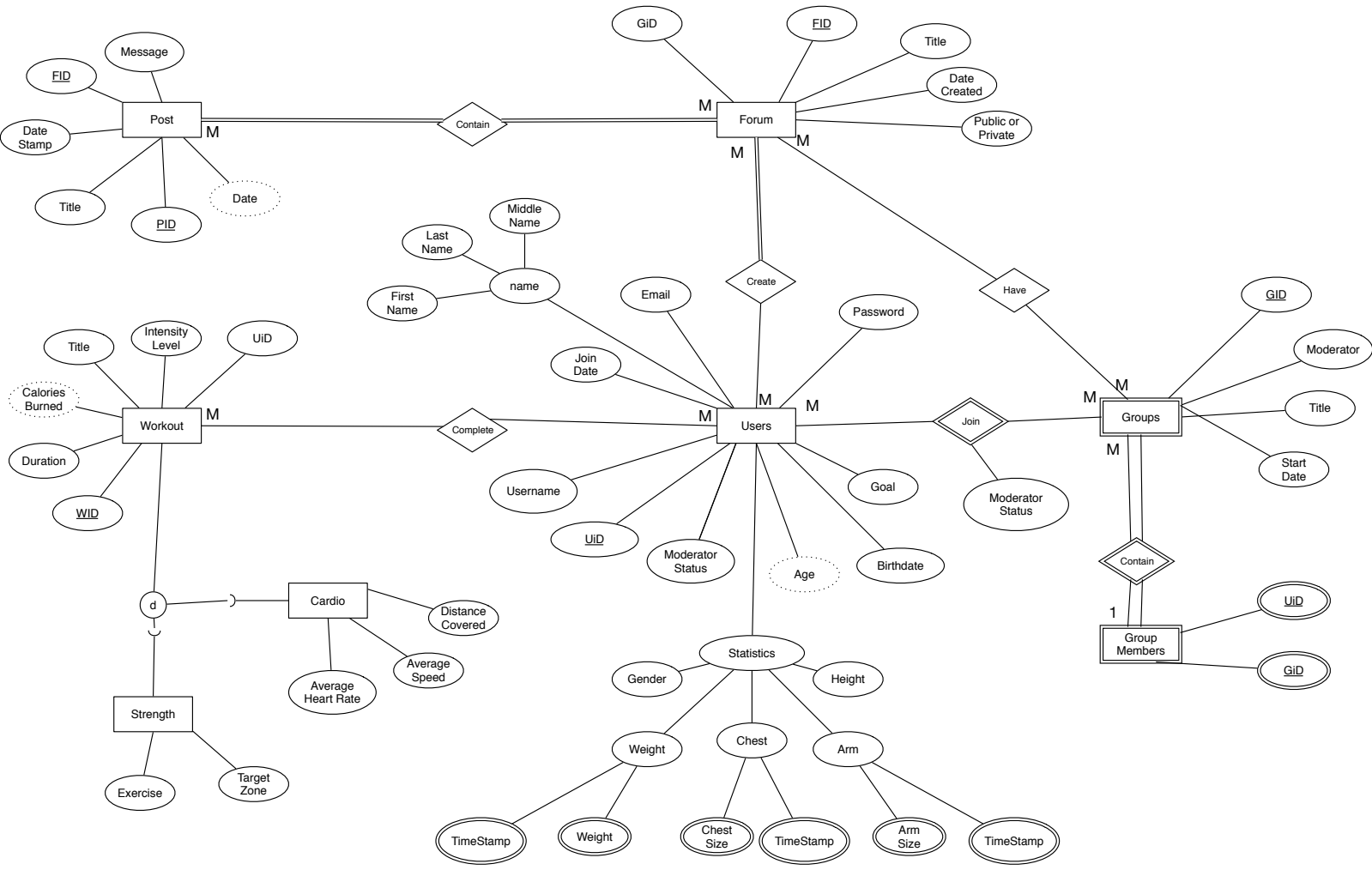
2015 marked a record year for Americans regularly exercising, hitting just over 55%. Keeping that momentum is difficult. As blue-collar jobs continue to decline, it has been more important than ever to keep a weekly regimen of healthy eating and exercising.

Living in the most interconnected generation poses quite a viable idea for social networking: healthy living. We can build a social network that connects users to friends, peers and family to make an online community of healthy living.

Our database will be essential because it will unite something so mundane and uninteresting with familiar faces. This will make exercise more enjoyable and offer a group to hold you accountable. We can shape an entire generation by having motivation a click away.

This database will consist of premade workouts, groups, and reminder emails to help you stay on top of your fitness plan.

2 Revised Conceptual Database Design



3 Logical Database Design

3.1 Relational Set

Please note primary keys are signified by **PK** and foreign keys are signified by *FK*.

Post

<u>PiD</u> PK	<u>FiD</u> <i>FK</i>	Message	DateStamp	Title	Date
----------------------	----------------------	---------	-----------	-------	------

Forum

<u>FiD</u> PK	Title	DateCreated	PublicOrPrivate	GiD <i>FK</i>
----------------------	-------	-------------	-----------------	---------------

Groups

<u>GiD</u> PK	Moderator <i>FK</i>	Title	StartDate
----------------------	---------------------	-------	-----------

Workouts

<u>WiD</u> PK	Duration	Title	IntensityLevel	CaloriesBurned	UiD <i>FK</i>
----------------------	----------	-------	----------------	----------------	---------------

Strength

<u>WiD</u> PK	Duration	Title	IntensityLevel	CaloriesBurned	UiD <i>FK</i>	Exercise	Target Zone
----------------------	----------	-------	----------------	----------------	---------------	----------	-------------

Cardio

<u>WiD</u> PK	Duration	Title	IntensityLevel	CaloriesBurned	UiD <i>FK</i>	AverageHeartRate
AverageSpeed	DistanceCovered					

Users

<u>UiD</u> PK	Username	Height	Birthdate	Goal	Password	JoinDate	Gender
FirstName	MiddleName	LastName					

Weight

Arm Size

<u>UiD</u> <i>PK, FK</i>	<u>TimeStamp</u> PK	Weight	<u>UiD</u> <i>PK, FK</i>	<u>TimeStamp</u> PK	Arm Size
--------------------------	----------------------------	--------	--------------------------	----------------------------	----------

Chest Size

<u>UiD</u> <i>PK, FK</i>	<u>TimeStamp</u> PK	ChestSize
--------------------------	----------------------------	-----------

Group Members

<u>UiD</u> PK	<u>GiD</u> <i>FK</i>	ModeratorStatus
----------------------	----------------------	-----------------

3.2 Summary Table

Attribute	Data Type	Constraints	Meaning
Message	varchar	500 characters	Contents of a post.
FID	int	unique	Forum ID.
DateStamp	timestamp	timestamp of creation	Timestamp of post was creation.
Title	char	35 characters	Title of a post.
PID	int	unique	Post ID.
Date	timestamp	date of creation	Timestamp of post creation.
Title	char	35 characters	Title of a forum.
DateCreated	timestamp	timestamp of creation	Timestamp of forum creation.
PublicOrPrivate	boolean	-	Is forum public.
GID	int	unique	Group ID.
Moderator	char	must match a username	Group owner/moderator.
Title	char	35 characters	Title of the group.
StartDate	timestamp	timestamp of creation	Timestamp of group creation.
IntensityLevel	int	-	Enumerated value, code for different options.
Title	char	35 characters	Title of the workout.
CaloriesBurned	int	num > 0	How many calories burned during exercise.
Duration	timestamp	num > 0	Length of exercise.
WID	int	unique	Workout ID.
Exercise	char	35 characters	Exercise name.
TargetZone	char	35 characters	Area exercise is intended to workout.
AverageHeartRate	int	num > 0	Average heart rate recorded during exercise.
AverageSpeed	int	num > 0	Average speed of cardio exercise.
DistanceCovered	int	num > 0	Distance covered during cardio exercise.
Password	char	-	Password of user.
JoinDate	timestamp	timestamp of creation	Timestamp of users sign up.
Username	char	unique	Users username, used for sign in.
ModeratorStatus	boolean	-	Is a moderator.
Birthdate	date	-	Date of users birth.
Goal	varchar	500 characters	Users intended workout goals.
Gender	char	1 character, M or F	Users gender, male(M) or female(F).
Weight	int	num > 0	Users weight at a certain date.
ChestSize	int	num > 0	Users chest size at a certain date.
ArmSize	int	num > 0	Users arm size at a certain date.
Height	int	num > 0	Users height at a certain date.
UID	int	unique	Users ID.
TimeStamp	date	unique	Date of users weight measurement.
TimeStamp	date	unique	Date of users chest measurement.
TimeStamp	date	unique	Date of users arm measurement

4 Application Program Design

Create a New User

This function creates a new user, accessing only the **user** table.

INPUT	Username, Height, Birthdate, Goal, Password, Gender.
STEPS	1. Check to see if the username is available. If available, proceed. If not, display appropriate message to notify the user. 2. Insert appropriate information to the User table. (Username to username, height to height) 3. Generate a user ID, assign it to the UiD attribute. 4. Take a time stamp, assign it to the JoinDate attribute. 5. Calculate the age based on the BirthDate attribute.
OUTPUT	A new User entity will be inserted into the table, with appropriate data into proper columns (along with computed properties and derived properties).
ASSUMPTIONS	Username, Height, Birthdate, Goal, Password, Gender are all correct (this will be validated in the sign up form).

Delete A Group

This function deletes a group by updating information in **Forum**, and removing the **Group** and **Group Members** tables.

INPUT	The Group ID (GiD) that is to be deleted.
STEPS	1. Check to see if the request is made by the moderator via the Moderator column in the Groups table. If true, approve the request. If not, cancel the request and notify the user. 2. Remove the row that has a matching GiD that was provided for deletion in the Groups table. 3. Query the Group Members table, removing any row that match the GiD provided for deletion. 4. Query the Forum table for any matching Group Ids (GiD), setting the GiD to null if matching.
OUTPUT	The groups are deleted, the group members within that group are deleted, and any reference to the group is deallocated.
ASSUMPTIONS	None.

Modifying User Statistics

Our user statistics have the ability to fluctuate. We would like to accommodate for this fluctuation by allowing users to update their respective statics; specifically, we would like to let users update **Username**, **Height**, **Goal**, **Password**, **Gender** in the **Users** table.

INPUT	The specific attribute(s) of the set Username , Height , Goal , Password , Gender that would like to be updated with the new value.
STEPS	1. Ensure the data is valid (e.g. is not null when applicable, in the proper domain). If it is valid, continue. If not, prompt the user with an error message and try again. 2. Modify the attribute to reflect the new value. 3. Repeat for any additional attributes provided.
OUTPUT	The attribute(s) should now reflect the new value provided.
ASSUMPTIONS	The data is within a proper range (will not overflow).

Query Other Users

This function allows for users to query other users; this can be done via **Username** or **FirstName**, **MiddleName**, and **LastName** from the **Users** table.

INPUT	Either a Username xor any subset of FirstName , MiddleName , or LastName .
STEPS	1. Check to see if input is valid. If is, proceed. If not, display error message to the user. 2. Query the Users table to see if the user exists. If the user exists, proceed. If not, display appropriate message to the user. 3. Project the profile.
OUTPUT	Either the search user will be projected or an error message is the user does not exist.
ASSUMPTIONS	The first, middle and last name are all provided. The names are unique (solely for the testing purposes).

User Leaderboards

Generate the leaderboard based on the workouts accomplished; specifically aggregating data from the **Strength** and **Cardio** table. *Note this is the function that requires multiple tables.*

INPUT	None.
STEPS	1. Merge the User and Workouts table, call the new table Merged. 2. Add up the total duration (call the new property TotalDuration) in the Merged table based on the UiD attribute, making a new table named Sums. 3. Sort the Sums by the TotalDuration attribute. 4. Display the top 10 on the sorted Sums table to the user. 5. Display the user their current rank.
OUTPUT	An eleven-row table displaying the top 10 leaderboards and the user's current rank.
ASSUMPTIONS	There is a bare minimum of eleven users.

5 User Interface Design

